

Middleware para Sistemas Distribuídos: Uma Análise do Eclipse Mosquitto

Otávio Miranda Côrtes
Universidade Federal de Uberlândia (UFU)
Monte Carmelo - MG, Brasil
otavio.cortes@ufu.br

Abstract—This paper presents a technical and practical study of Eclipse Mosquitto, an open-source MQTT broker widely used in distributed systems and Internet of Things (IoT) applications. The study explores the operation, architecture, and main features of Mosquitto as a message-oriented middleware, based on the publish/subscribe communication model. Through conceptual review, configuration analysis and hands-on testing on Windows, this work simulates a distributed system using temperature sensors to validate how Mosquitto facilitates asynchronous and reliable communication between distributed nodes. The paper also discusses the Quality of Service (QoS) levels, retained messages, session persistence, and the configuration flexibility of the mosquitto.conf file, demonstrating the broker's applicability in real-world distributed IoT scenarios.

Index Terms—Sistemas Distribuídos, Middleware, Eclipse Mosquitto, MQTT, Internet das Coisas, QoS, Publish/Subscribe

RESUMO

Este artigo apresenta um estudo técnico e prático sobre o Eclipse Mosquitto, um broker MQTT de código aberto amplamente utilizado em sistemas distribuídos e aplicações de Internet das Coisas (IoT). O estudo aborda a operação, arquitetura e principais funcionalidades do Mosquitto como middleware orientado a mensagens, baseado no modelo de comunicação publish/subscribe. Através de revisão conceitual, análise de configuração e testes práticos realizados no Windows, simula-se um sistema distribuído com sensores de temperatura para validar como o Mosquitto facilita a comunicação assíncrona e confiável entre nós distribuídos. Também são abordados os níveis de Qualidade de Serviço (QoS), mensagens retidas, persistência de sessão e a flexibilidade de configuração por meio do arquivo mosquitto.conf, demonstrando sua aplicabilidade em cenários reais de IoT.

I. INTRODUÇÃO

A crescente demanda por sistemas interconectados, escaláveis e tolerantes a falhas impulsionou o desenvolvimento e a adoção de Sistemas Distribuídos nas mais diversas áreas da computação. Um sistema distribuído é definido como “um conjunto de computadores independentes que se apresenta aos usuários como um sistema único e coerente”, conforme Tanenbaum e van Steen [1]. Essa arquitetura oferece vantagens como compartilhamento de recursos, escalabilidade, resiliência e desempenho, mas também traz desafios relacionados à comunicação, sincronização, segurança e heterogeneidade.

Nesse contexto, torna-se essencial o uso de soluções intermediárias que facilitem a comunicação entre os componentes

distribuídos. Segundo a Red Hat, middleware é “uma camada de software que conecta o sistema operacional a aplicações, dados e usuários” [2], provendo mecanismos padronizados para troca de mensagens, descoberta de serviços, autenticação, entre outros.

Dentre os diversos tipos de middleware existentes, os orientados a mensagens têm se destacado, especialmente em aplicações de Internet das Coisas (IoT), onde há necessidade de comunicação leve, assíncrona e eficiente entre dispositivos. O protocolo MQTT (Message Queuing Telemetry Transport) e o broker Eclipse Mosquitto têm ganhado relevância nesse cenário por sua leveza, confiabilidade e ampla adoção em ambientes com restrições de banda, energia e processamento [3].

Este trabalho tem como objetivo apresentar testes que demonstrem como o middleware Eclipse Mosquitto opera e contribui para o desenvolvimento de um sistema distribuído, abordando seus fundamentos técnicos, arquitetura operacional e aplicação prática em um ambiente distribuído.



Fig. 1. Middleware Eclipse Mosquitto atuando como broker MQTT na comunicação entre diversos dispositivos conectados em uma rede IoT. Fonte: adaptado de [4].

II. FUNDAMENTAÇÃO

O entendimento do funcionamento do Eclipse Mosquitto exige a compreensão prévia de conceitos fundamentais relacionados a sistemas distribuídos, em especial aqueles associados

a middlewares orientados a mensagens, brokers, o protocolo MQTT, Internet das Coisas (IoT) e níveis de Qualidade de Serviço (QoS). Tais conceitos estão intrinsecamente ligados ao tema deste trabalho e serão referenciados ao longo de toda a sua extensão. Esta seção tem como objetivo apresentar esses elementos de forma estruturada, descrevendo suas definições, propósitos e funcionamento básico, a fim de estabelecer uma base teórica sólida para a análise do middleware Eclipse Mosquitto.

A. Middlewares orientados a mensagens

Middleware Orientado a Mensagens, ou MOM (Message-Oriented Middleware), é uma categoria de software intermediário projetada para permitir a comunicação assíncrona entre componentes distribuídos, mesmo quando esses utilizam diferentes protocolos de mensagens. Sua principal característica é o desacoplamento entre os emissores (publishers) e os receptores (subscribers), o que significa que eles não precisam interagir diretamente ou estar ativos ao mesmo tempo para que a troca de informações ocorra.

Segundo a IBM, o MOM não apenas gerencia o envio e recebimento de mensagens entre aplicações, mas também realiza o roteamento dessas mensagens, garantindo que cheguem corretamente aos destinatários apropriados e, quando necessário, que sejam entregues na ordem correta [5]. Além disso, pode traduzir ou transformar o conteúdo das mensagens para facilitar a interoperabilidade entre sistemas heterogêneos.

B. Brokers

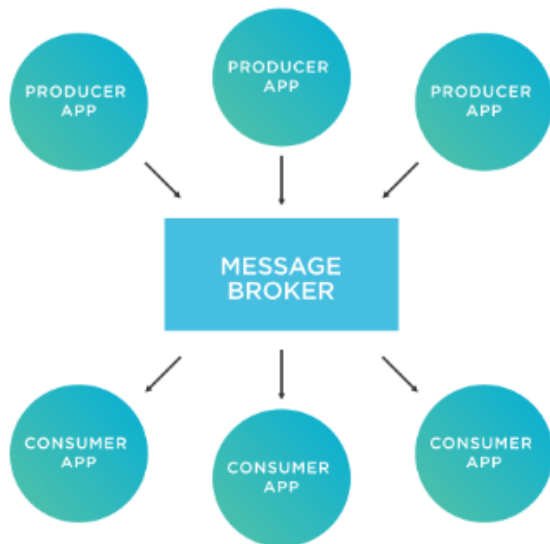


Fig. 2. Funcionamento de um message broker: os produtores enviam mensagens para o broker, que as encaminha aos consumidores. Fonte: adaptado de [6].

Em sistemas distribuídos baseados em mensagens, o broker (ou corretor de mensagens) é um componente central que atua como intermediário entre os emissores (publishers) e os receptores (subscribers) de mensagens. Sua função principal é

receber, processar, armazenar temporariamente (se necessário) e encaminhar as mensagens publicadas para os assinantes corretos, com base em critérios como tópicos, assinaturas e políticas de entrega.

O uso de brokers promove o desacoplamento total entre os participantes da comunicação. Os emissores não precisam conhecer a identidade, localização ou disponibilidade dos receptores, e vice-versa. Esse modelo reduz o acoplamento entre os componentes do sistema, melhora a escalabilidade e simplifica a adição ou remoção de novos nós [7]. As principais responsabilidades de um broker incluem:

- Gerenciar conexões com clientes produtores e consumidores de mensagens;
- Realizar o roteamento das mensagens com base em tópicos ou regras de filtragem;
- Aplicar políticas de Qualidade de Serviço (QoS), garantindo confiabilidade na entrega;
- Armazenar mensagens temporariamente para sessões persistentes ou mensagens retidas;
- Implementar mecanismos de autenticação e autorização, assegurando controle de acesso;
- Monitorar e gerenciar o estado das conexões ativas.

No modelo de comunicação publish/subscribe, o broker é um elemento obrigatório, pois centraliza o fluxo das mensagens e garante que os dados cheguem aos assinantes interessados. Sua atuação permite a comunicação assíncrona e distribuída entre os nós, mesmo em ambientes com conectividade limitada ou intermitente.

C. Protocolo MQTT

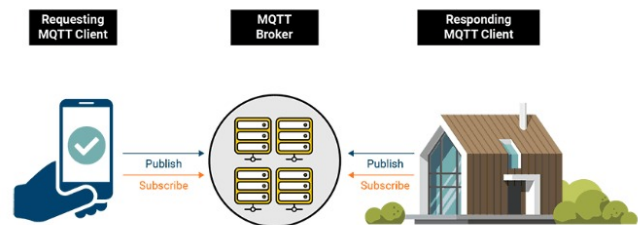


Fig. 3. Funcionamento do protocolo MQTT: um cliente requisitante publica dados para o broker, que os entrega ao cliente que está inscrito no tópico. Fonte: adaptado de [3].

“MQTT é um protocolo de transporte de mensagens de publicação/assinatura cliente-servidor. É leve, aberto, simples e projetado para ser fácil de implementar. Essas características o tornam ideal para uso em diversas situações, incluindo ambientes com restrições, como comunicação em contextos de Máquina a Máquina (M2M) e Internet das Coisas (IoT), onde um pequeno espaço de código é necessário e/ou a largura de banda da rede é limitada” [3].

Essa leveza e simplicidade tornam o MQTT particularmente adequado para sistemas embarcados, sensores remotos e dispositivos que operam com conectividade instável. Diferente dos modelos tradicionais de comunicação cliente-servidor, o

MQTT opera de forma assíncrona e baseada em tópicos, permitindo que os dispositivos publiquem e assinem mensagens de forma desacoplada.

Além de suportar Qualidade de Serviço (QoS) configurável, o MQTT permite:

- **Mensagens retidas:** o broker pode armazenar a última mensagem de um tópico e entregá-la automaticamente a novos assinantes assim que se conectarem;
- **Sessões persistentes:** mensagens destinadas a clientes offline podem ser armazenadas temporariamente e enviadas após a reconexão, garantindo que dados não sejam perdidos;
- **Tópicos hierárquicos e uso de curingas:** facilitam o controle fino sobre o tráfego de mensagens;
- **Controle de acesso e segurança:** através de autenticação, criptografia TLS/SSL e listas de controle de acesso (ACLs).

D. Internet das Coisas (IoT)



Fig. 4. Conceito de Internet das Coisas (IoT), conectando dispositivos físicos à rede para troca de dados e automação de processos. Fonte: adaptado de [8].

A Internet das Coisas (Internet of Things – IoT) é um paradigma tecnológico que permite que objetos físicos, como sensores, máquinas, veículos e dispositivos domésticos, estejam conectados à internet e possam coletar, processar e trocar dados automaticamente. Essa conectividade permite a automação e o monitoramento em tempo real de ambientes físicos, contribuindo para a tomada de decisões inteligentes e a otimização de processos em diferentes setores.

Segundo a Oracle, a IoT representa uma rede de dispositivos interconectados que se comunicam continuamente por meio de sensores, software e outras tecnologias, capturando dados do mundo físico e enviando-os para sistemas digitais capazes de interpretar e reagir a essas informações. [9].

Aplicações de IoT incluem desde casas inteligentes, controle industrial e agricultura de precisão até cidades inteligentes, logística e saúde conectada. Em todos esses contextos, os dispositivos precisam de uma forma eficiente, confiável e leve de se comunicar mesmo em ambientes com largura de banda limitada ou intermitência de conexão.

Nesse cenário, protocolos como o MQTT e brokers como o Eclipse Mosquitto desempenham papel fundamental, possibilitando a troca de mensagens entre os dispositivos de forma assíncrona, com baixo consumo de recursos e suporte a qualidade de serviço. A arquitetura publish/subscribe, comum em sistemas IoT, contribui para o desacoplamento entre emissores e receptores, permitindo maior escalabilidade e flexibilidade na rede.

E. Qualidade de Serviço (QoS)

A Qualidade de Serviço (QoS – Quality of Service) é um dos principais recursos do protocolo MQTT, permitindo que o desenvolvedor escolha o nível de confiabilidade com que as mensagens são entregues entre publicadores e assinantes. O MQTT define três níveis de QoS: 0, 1 e 2, cada um oferecendo diferentes garantias de entrega, com implicações diretas sobre consumo de recursos, latência e tráfego de rede.

1) *QoS 0 – “No máximo uma vez”*: Neste nível, a mensagem é enviada uma única vez e não há confirmação de entrega. Se a mensagem se perder durante a transmissão, ela não será reenviada. Esse é o nível mais leve, ideal para aplicações em que eventuais perdas de mensagens não comprometem o funcionamento do sistema, como em leituras periódicas de sensores onde a última medição é mais relevante que o histórico completo.

2) *QoS 1 – “Pelo menos uma vez”*: Com QoS 1, o publicador envia a mensagem ao broker e aguarda uma confirmação (acknowledgment). Caso o broker não responda dentro de um intervalo de tempo, a mensagem é reenviada até que a confirmação seja recebida. Isso garante que a mensagem será entregue, mas pode haver duplicações no lado do assinante. É adequado para cenários em que a perda de mensagens é inaceitável, mas mensagens duplicadas podem ser toleradas, como no envio de comandos a dispositivos de controle.

3) *QoS 2 – “Exatamente uma vez”*: Este é o nível mais confiável oferecido pelo MQTT, garantindo que a mensagem será entregue uma única vez e sem duplicações. Para isso, o protocolo realiza um handshake em quatro etapas entre publicador, broker e assinante. O broker armazena a mensagem até que a confirmação definitiva seja recebida. Por ser mais complexo, esse nível exige mais memória, processamento e troca de pacotes. É recomendado para aplicações críticas, como transações financeiras, alarmes ou notificações que não podem ser perdidas nem duplicadas.

4) *Considerações sobre o uso de QoS*: A escolha do nível de QoS adequado depende diretamente dos requisitos da aplicação. Níveis mais altos oferecem maior confiabilidade, mas também geram maior sobrecarga de rede e latência. É essencial equilibrar a confiabilidade e os recursos disponíveis no ambiente, especialmente em dispositivos com restrições de banda, energia ou capacidade de processamento. [3]

III. OPERAÇÃO

O Eclipse Mosquitto é um middleware orientado a mensagens que atua como um broker MQTT leve, eficiente e de código aberto. Seu papel central é gerenciar a comunicação

entre múltiplos dispositivos distribuídos que utilizam o protocolo MQTT, adotando uma arquitetura baseada no modelo publish/subscribe. Esta seção aprofunda-se na arquitetura, no fluxo de funcionamento, nos recursos avançados e nos serviços oferecidos pelo Mosquitto, explicando como ele viabiliza a interoperabilidade entre componentes de um sistema distribuído de forma assíncrona e confiável [10].

A. Principais Recursos do Mosquitto

Segundo a EMQX [11], seus principais diferenciais incluem:

- **Baixo consumo de recursos:** o Mosquitto foi projetado em C para ter uma pegada leve, com baixíssimo uso de CPU e memória. Isso o torna ideal para execução em dispositivos embarcados e ambientes com recursos limitados, como Raspberry Pi, roteadores ou gateways IoT.
- **Compatibilidade com o padrão MQTT:** o Mosquitto implementa totalmente o protocolo MQTT 3.1, 3.1.1 e 5.0, permitindo interoperabilidade com qualquer cliente MQTT compatível. Isso facilita a integração com sensores, atuadores, serviços em nuvem e painéis de visualização.
- **Multiplataforma:** disponível para diversas arquiteturas e sistemas operacionais, incluindo Linux, Windows, macOS, e plataformas ARM. Essa portabilidade garante que possa ser usado tanto em ambiente local quanto em produção.
- **Sessões persistentes e mensagens retidas:** o Mosquitto oferece suporte à persistência de sessões de clientes e à retenção de mensagens, garantindo que informações críticas sejam entregues mesmo após desconexões temporárias dos dispositivos.
- **Controle de acesso via ACLs:** permite configurar políticas de autorização por usuário e tópico, assegurando que os dados sejam acessados somente por clientes devidamente autorizados.
- **Autenticação personalizada:** além do suporte nativo a usuários e senhas por arquivo, pode ser estendido para autenticação via plugins externos ou integração com sistemas próprios.
- **Conexões seguras (TLS/SSL):** o Mosquitto suporta comunicação criptografada, com autenticação mútua baseada em certificados digitais. Isso é essencial para aplicações em redes públicas ou com exigências de segurança.
- **Mensagens com QoS configurável:** permite escolher entre três níveis de qualidade de serviço (0, 1 e 2), equilibrando confiabilidade e desempenho conforme as necessidades da aplicação.
- **Modo bridge (ponte entre brokers):** possibilita a interligação de múltiplas instâncias do Mosquitto para replicação de tópicos, distribuição de carga ou replicação de dados entre ambientes locais e a nuvem.
- **Facilidade de instalação e uso:** com instalação simples e arquivos de configuração bem documentados, é uma ex-

celente escolha para prototipagem rápida e para usuários iniciantes no ecossistema MQTT.

B. Modelo de Funcionamento do Mosquitto

O funcionamento básico do Mosquitto segue o modelo publish/subscribe:

- 1) Clientes se conectam ao broker via TCP, WebSockets ou TLS.
- 2) Clientes publicam mensagens em tópicos (PUBLISH).
- 3) O broker identifica todos os clientes assinantes daquele tópico (SUBSCRIBE) e repassa a mensagem, conforme as regras de QoS, ACLs e autenticação.
- 4) Caso haja clientes desconectados com sessão persistente, o broker armazena a mensagem até a reconexão.

C. Configurações Fundamentais do Mosquitto

A configuração do Mosquitto é feita no arquivo `mosquitto.conf`, que pode ser carregado no momento da execução do broker. De acordo com o manual oficial da ferramenta [12] os principais parâmetros, que se destacam são:

1) *Listeners e Porta de Comunicação:* Define as portas nas quais o Mosquitto aceitará conexões:

```
listener 1883
```

2) *Autenticação e ACLs:* Permite configurar login e senha para os clientes, e aplicar restrições por tópico:

```
allow_anonymous false
password_file /etc/mosquitto/passwd
acl_file /etc/mosquitto/acl
```

As ACLs controlam quem pode publicar ou assinar tópicos específicos:

```
user sensor1
topic write sensores/temperatura
topic read comandos/#
```

3) *Persistência:* A persistência pode ser habilitada para armazenar sessões e mensagens entre reinicializações:

```
persistence true
persistence_location /var/lib/mosquitto/
```

4) *Mensagens Retidas:* Mensagens retidas garantem que a última mensagem publicada em um tópico esteja disponível para novos assinantes:

```
mosquitto_pub -t "configuracao/luz" -m
"on" -r
```

Isso garante que clientes com sessões duráveis recuperem mensagens após quedas de energia ou reboot do broker.

5) *Limites e Desempenho:* O Mosquitto permite configurar limites de mensagens por segundo, quantidade de conexões simultâneas e tamanho máximo de payload:

```
max_inflight_messages 20
max_queued_messages 1000
message_size_limit 1048576
```

6) *Segurança e TLS*: O Mosquitto suporta criptografia via TLS/SSL e autenticação por certificado, permitindo conexões seguras em ambientes corporativos ou públicos. Os parâmetros cafile, certfile e keyfile devem ser configurados no listener.

A operação do Mosquitto é altamente personalizável por meio do arquivo mosquitto.conf, o que o torna adequado para uma ampla gama de aplicações desde pequenos dispositivos embarcados até infraestruturas IoT complexas. Ao entender suas capacidades configuráveis, é possível ajustar o desempenho, a segurança e a confiabilidade do broker conforme as necessidades do sistema distribuído.

IV. TESTE

Esta seção apresenta um guia passo a passo para instalação, execução e testes com o middleware Eclipse Mosquitto em um ambiente Windows. O objetivo é simular um sistema distribuído de Internet das Coisas (IoT), o cenário consiste em sensores de temperatura que publicam dados para um tópico, e consumidores que os recebem. Durante os testes, também serão demonstradas funcionalidades como QoS e persistência.

A. Instalação do Mosquitto no Windows

- 1) Acesse o site oficial:
`https://mosquitto.org/download.`
- 2) Baixe a versão **Windows (Installer)** mais recente.
Exemplo: `mosquitto-2.0.18-install-windows-x64.exe`
- 3) Durante a instalação:
 - Marque a opção para instalar o Mosquitto como um serviço do Windows.
 - Marque também a opção de instalar as ferramentas CLI (`mosquitto_pub` e `mosquitto_sub`).
- 4) **Inicie o serviço (execute como Administrador):**
`net start mosquitto`

B. Verificação do Serviço

Abra o terminal (CMD) e execute:

```
netstat -an | findstr 1883
```

Se o serviço estiver ativo, a porta 1883 estará em escuta (LISTENING), indicando que o broker está rodando.

Após a instalação, o Mosquitto será iniciado automaticamente como serviço na porta 1883.

C. Simulação de Sistema Distribuído com Sensores de Temperatura

O cenário simula:

- Um sensor publicando dados de temperatura em um tópico MQTT.
- Um sistema consumidor que assina o tópico e recebe os dados em tempo real.

1) *Assinatura de um tópico (dispositivo consumidor)*: Abra o terminal (CMD) e digite:

```
mosquitto_sub -h localhost -t  
"iot/temperatura" -v
```

Este comando escuta o tópico e exibe as mensagens recebidas. Esse terminal representa o consumidor do sistema distribuído.

2) *Publicação de mensagens (sensor IoT)*: Em outra janela de terminal, simule a publicação de valores com diferentes configurações:

a) *Publicação simples (sem QoS/persistência)*:

```
mosquitto_pub -h localhost -t  
"iot/temperatura" -m "25.3"
```

b) *Publicação com QoS 1 (entrega garantida pelo menos uma vez)*:

```
mosquitto_pub -h localhost -t  
"iot/temperatura" -m "26.1" -q 1
```

c) *Publicação com QoS 2 (entrega garantida exatamente uma vez)*:

```
mosquitto_pub -h localhost -t  
"iot/temperatura" -m "26.9" -q 2
```

d) *Publicação com persistência (mensagem retida)*:

```
mosquitto_pub -h localhost -t  
"iot/temperatura" -m "24.8" -r
```

3) *Verificando a persistência*: Após publicar com a flag `-r`, feche o terminal assinante. Em seguida, abra uma nova janela e assine novamente o tópico:

```
mosquitto_sub -h localhost -t  
"iot/temperatura" -v
```

O terminal mostrará imediatamente a última mensagem retida:

```
iot/temperatura 24.8
```

D. Considerações finais dos testes

Os testes realizados permitiram observar na prática mesmo que de maneira simulada o papel fundamental do Eclipse Mosquitto como middleware em sistemas distribuídos. A instalação e execução em ambiente Windows demonstraram a facilidade de configuração do broker, enquanto a simulação de sensores de temperatura com publicação e assinatura de mensagens evidenciou o funcionamento do modelo publish/subscribe de maneira simples e didática.

Foi possível validar, de forma clara, os principais recursos oferecidos pelo Mosquitto:

- O gerenciamento de tópicos MQTT, permitindo múltiplos dispositivos trocarem mensagens de forma assíncrona e desacoplada.
- A aplicação dos níveis de Qualidade de Serviço (QoS), demonstrando diferentes garantias de entrega de mensagens.

- A funcionalidade de persistência por meio da retenção de mensagens, útil em sistemas distribuídos que exigem estado inicial ou últimas leituras.

Essa simulação básica reflete situações reais encontradas em sistemas de Internet das Coisas (IoT), onde sensores e atuadores precisam se comunicar de forma leve, confiável e eficiente. Mesmo demonstrando poucas funcionalidades do Mosquitto podemos identificar que se trata de uma solução robusta e adequada para esse tipo de arquitetura, cumprindo seu papel como middleware orientado a mensagens.

V. CONSIDERAÇÕES FINAIS

O estudo apresentado demonstrou, de forma teórica e prática, como o middleware Eclipse Mosquitto pode ser utilizado em sistemas distribuídos para promover comunicação leve, confiável e eficiente entre dispositivos. Com base em uma fundamentação sólida sobre middleware orientado a mensagens, brokers MQTT, protocolo MQTT e aplicações em Internet das Coisas (IoT), foi possível compreender o papel que o Mosquitto desempenha ao permitir a troca assíncrona de mensagens entre nós distribuídos.

A seção de operação mostrou que o Mosquitto é uma solução robusta, personalizável e compatível com ambientes de produção, oferecendo suporte a segurança, persistência, controle de acesso e QoS. Por fim, os testes práticos simularam um cenário real de sensores de temperatura publicando dados em tópicos, com consumidores recebendo essas informações em tempo real, validando o funcionamento completo do broker.

Apesar de sua simplicidade, o Mosquitto atende muito bem aos requisitos de sistemas distribuídos modernos, sendo especialmente indicado para aplicações de IoT e automação. Como trabalho futuro, recomenda-se a implementação de um sistema completo com múltiplos sensores e múltiplos consumidores, além da integração com ferramentas de visualização de dados ou bancos de dados, para persistência de longo prazo.

REFERENCES

- [1] A. A. Loper, "Conceitos e características dos sistemas distribuídos," *Projetos de redes e sistemas distribuídos*, p. 9.
- [2] Red Hat. (2024) O que é middleware? Acessado em: 24 jul. 2025. [Online]. Available: <https://www.redhat.com/pt-br/topics/middleware/what-is-middleware>
- [3] HiveMQ Team. (2014) Mqtt essentials part 1: Introducing mqtt. Acessado em: 24 jul. 2025. [Online]. Available: <https://www.hivemq.com/blog/mqtt-essentials-part-1-introducing-mqtt/>
- [4] iMasters. (2022) Servidor eclipse mosquitto mqtt na nuvem: como criar uma rede iot pessoal. Acessado em: 24 jul. 2025. [Online]. Available: <https://imasters.com.br/cloud/servidor-eclipse-mosquitto-mqtt-na-nuvem-como-criar-uma-rede-iot-pessoal>
- [5] IBM Corporation. (2024) O que é middleware? Acessado em: 25 jul. 2025. [Online]. Available: <https://www.ibm.com/br-pt/topics/middleware>
- [6] Remsoft. (2022) O que é message broker? como o uso de mensageria pode contribuir em seu projeto. Acessado em: 24 jul. 2025. [Online]. Available: <https://remsoft.com.br/blog/tecnologias/o-que-e-message-broker-como-o-uso-de-mensageria-pode-contribuir-em-seu-projeto/>
- [7] IBM Corporation. (2024) O que é um broker de mensagens? Acessado em: 25 jul. 2025. [Online]. Available: <https://www.ibm.com/br-pt/topics/message-brokers>
- [8] ComCiência. (2019) Internet das coisas coloca dados e privacidade em risco. Acessado em: 24 jul. 2025. [Online]. Available: <https://www.comciencia.br/internet-das-coisas-coloca-dados-e-privacidade-em-risco/>

- [9] Oracle Corporation. (2024) O que é a internet das coisas (iot)? Acessado em: 26 jul. 2025. [Online]. Available: <https://www.oracle.com/br/internet-of-things/>
- [10] Eclipse Foundation. (2024) Eclipse mosquitto - an open source mqtt broker. Acessado em: 24 jul. 2025. [Online]. Available: <https://mosquitto.org/>
- [11] EMQ Technologies Co., Ltd. (2024) Mosquitto mqtt broker: Pros, cons, tutorial, and modern alternatives. Acessado em: 24 jul. 2025. [Online]. Available: <https://www.emqx.com/en/blog/mosquitto-mqtt-broker-pros-cons-tutorial-and-modern-alternatives>
- [12] Eclipse Foundation. (2024) mosquitto.conf – mosquitto configuration manual (v2.0). Acessado em: 24 jul. 2025. [Online]. Available: <https://mosquitto.org/man/mosquitto-conf-5.html>